

Path Replanning

Path Replanning

1. Course Content
2. Preparation
3. Path Replanning Demonstration Case
4. Source Code Analysis
5. Parameter Debugging

1. Course Content

Basic

- Start the road network navigation feature and learn to use the `roadnet_nav` visualization panel to send target waypoints. Place obstacles on the navigation path to block the current edge; the robot will back up to the last safe waypoint it passed and replan a feasible path to reach the target waypoint

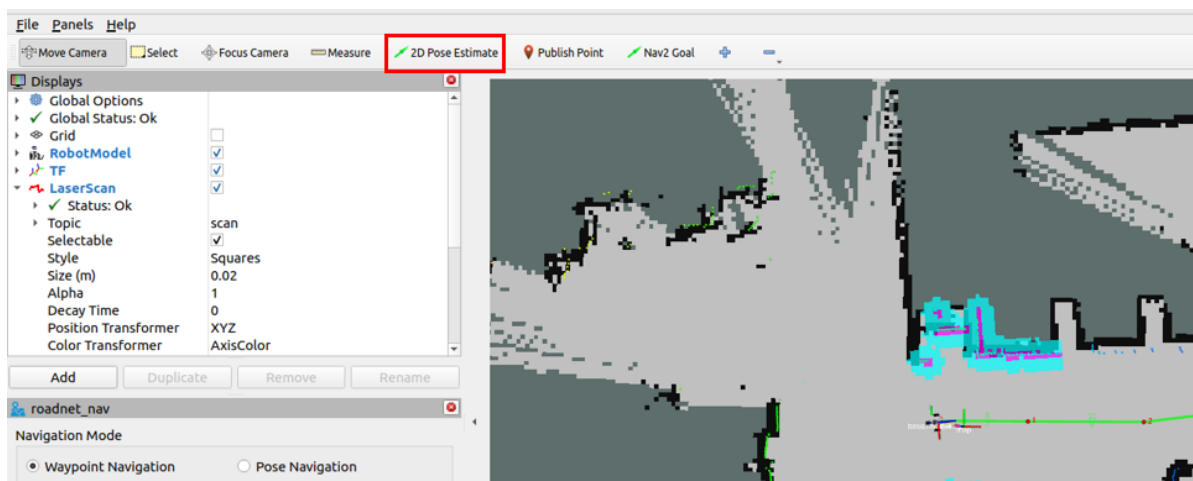
Advanced

- Understand the source code implementation logic
- Learn to configure the parameter file, including enabling and disabling the path replanning feature and adjusting the collision detection distance

2. Preparation

```
ros2 launch microros_v2_bringup roadnet_nav.launch.py
```

- Click **2D Pose Estimate** to initialize the robot's pose on the map



3. Path Replanning Demonstration Case

- Here we take waypoint-to-waypoint navigation as an example: navigate from waypoint 3 through the road network to waypoint 5, and place obstacles in advance on the road passed in between



- When an obstacle is detected within 0.5 meters in front of **base_link**, path replanning is triggered, and an alternative path 3—>4—>5 is selected to reach the target point.
- Obstacles are displayed and marked on the map as red spheres.



4. Source Code Analysis

- Source code path of the implementation program:

```
$HOME/mircoros_ws/src/roadnet/src/route_server.cpp
```

(Explain the program here - to be added)

```

void RouteServer::computeRouteAndFollow()
{
    RCLCPP_INFO(get_logger(), "Computing route and following path to goal.");

    auto goal = compute_route_and_follow_server->get_current_goal();
    auto result = std::make_shared<RoadNetToGoalResult>();
    auto feedback = std::make_shared<RoadNetToGoalFeedback>();
    ReroutingState rerouting_info;
    auto start_time = this->now();

    try
    {
        Route route;
        nav_msgs::msg::Path path;
        // Replanning trigger condition: (obstacle detection) or a replanning request
        (ReroutingService)
        while (rclcpp::ok())
        {
            if (!isRequestValid(compute_route_and_follow_server_))
            {
                return;
            }
            // Check whether a new goal has preempted the current one
            if (compute_route_and_follow_server->is_preempt_requested())
            {
                RCLCPP_INFO(get_logger(), "A new request is received, and the target
of the original request is preempted");
                goal = compute_route_and_follow_server->accept_pending_goal();
                rerouting_info.reset();
            }
            // Clear residual cost values from the global_costmap before planning to
            avoid affecting the plan
            clearGlobalCostmap();

            // Compute the feasible path in the road network
            route = findRoute(goal, rerouting_info);
            RCLCPP_INFO(get_logger(), "Route found with %zu nodes and %zu edges",
                route.edges.size() + 1u, route.edges.size());
            path = path_converter->densify(route, rerouting_info, route_frame_,
this->now());

            // Print the path
            std::string route_str;
            if (route.start_node)
            {
                route_str = std::to_string(route.start_node->nodeid);
            }
            for (const auto &edge : route.edges)
            {
                route_str += " ----->> " + std::to_string(edge->end->nodeid);
            }
            RCLCPP_INFO(get_logger(), "Find route plan: %s", route_str.c_str());

            // wait for FollowPath action server
            if (!follow_path_client_-
>wait_for_action_server(std::chrono::seconds(5)))
            {
                RCLCPP_ERROR(get_logger(), "FollowPath action server not available!");
            }
        }
    }
}

```

```

        result->error_code = RoadNetToGoalResult::UNKNOWN;
        result->error_msg = "FollowPath action server not available";
        compute_route_and_follow_server->terminate_current(result);
        return;
    }
    // Send the path to FollowPath
    auto follow_goal = FollowPath::Goal();
    follow_goal.path = path;

    auto send_goal_options =
rclcpp_action::Client<FollowPath>::SendGoalOptions();
    auto goal_handle_future = follow_path_client_-
>async_send_goal(follow_goal, send_goal_options);

    // wait for goal acceptance
    if (goal_handle_future.wait_for(std::chrono::seconds(5)) !=
std::future_status::ready)
    {
        RCLCPP_ERROR(get_logger(), "FollowPath goal send timed out!");
        result->error_code = RoadNetToGoalResult::UNKNOWN;
        result->error_msg = "FollowPath goal send timed out";
        compute_route_and_follow_server->terminate_current(result);
        return;
    }

    auto follow_goal_handle = goal_handle_future.get();
    if (!follow_goal_handle)
    {
        RCLCPP_ERROR(get_logger(), "FollowPath goal was rejected!");
        result->error_code = RoadNetToGoalResult::UNKNOWN;
        result->error_msg = "FollowPath goal rejected by controller";
        compute_route_and_follow_server->terminate_current(result);
        return;
    }
    RCLCPP_DEBUG(get_logger(), "FollowPath goal accepted, robot is
moving!");

    // Get result future for polling
    auto follow_result_future = follow_path_client_-
>async_get_result(follow_goal_handle);

    // Track path progress and detect obstacles
    double radius_threshold = 2.0;
    nav2_util::declare_parameter_if_not_declared(
        shared_from_this(), "radius_to_achieve_node",
        rclcpp::ParameterValue(radius_threshold));
    radius_threshold = get_parameter("radius_to_achieve_node").as_double();

    EdgePtr current_edge = nullptr;
    NodePtr next_node = nullptr;
    int edge_idx = -1;
    RouteTrackingState tracking_state;
    tracking_state.last_node = route.start_node;
    tracking_state.next_node = route.start_node;

    // Advance to first edge
    if (!route.edges.empty())
    {

```

```

    edge_idx = 0;
    current_edge = route.edges[0];
    next_node = current_edge->end;
    tracking_state.current_edge = current_edge;
    RCLCPP_INFO(get_logger(),
                "Entering edge:%u id:%u -----> id:%u",
                current_edge->edgeid,
                current_edge->start->nodeid,
                current_edge->end->nodeid);
}
rclcpp::Rate rate(20.0);
bool follow_completed = false;
bool route_tracking_done = route.edges.empty();
bool need_reroute = false;

while (rclcpp::ok() && !follow_completed && !need_reroute)
{
    // Check if our action was cancelled
    if (compute_route_and_follow_server->is_cancel_requested())
    {
        RCLCPP_INFO(get_logger(), "Goal cancelled, cancelling
FollowPath...");
        follow_path_client->async_cancel_goal(follow_goal_handle);
        compute_route_and_follow_server->terminate_all();
        return;
    }
    // Check if preempted by a new goal
    if (compute_route_and_follow_server->is_preempt_requested())
    {
        RCLCPP_INFO(get_logger(), "Goal preempted, cancelling
FollowPath...");
        follow_path_client->async_cancel_goal(follow_goal_handle);
        break;
    }
}
.....

```

5. Parameter Debugging

- Path of the road network navigation parameter file:

```
$HOME/mircoros_ws/src/microros_v2_bringup/config/roadnet_nav.yaml
```

(Explain here how to modify the parameters to enable and disable the replanning feature and to disable the CollisionMonitor - note that if it is disabled, navigation will have no obstacle avoidance; and how to adjust the collision detection distance)

```

route_server:
  ros__parameters:
    # Path generation
    route_frame: "map"
    path_density: 0.05

    tracker_update_rate: 3.0
    aggregate_blocked_ids: false

```

```

    boundary_radius_to_achieve_node: 0.3          #Achievement radius threshold
for the path (start / end point)
    radius_to_achieve_node: 0.35                  #Achievement radius threshold
for intermediate path nodes (neither start nor end point)
    # Nearest neighbor search
    enable_nn_search: false                       #whether to enable costmap-based
breadth-first search to find the nearest node (when disabled, only the nearest
waypoint by Euclidean distance is used; when enabled, collisions along the path
are considered)
    max_nn_search_iterations: 10000
    num_nearest_nodes: 5                         #Number of nearest neighbor nodes found
with the KD-tree

    smooth_corners: false                        #Whether to enable corner
smoothing
    #smoothing_radius: 0.2
    #smoothing_angle_threshold: 2.96 # Angle smoothing threshold

    # Path pruning
    max_prune_dist_from_edge: 8.0
    min_prune_dist_from_goal: 0.15
    min_prune_dist_from_start: 0.10
    prune_goal: true

    # Rerouting service
    operations: ["ReroutingService", "CollisionMonitor"]
    # operations: ["ReroutingService"]
    ReroutingService:
        plugin: "roadnet::ReroutingService"
    CollisionMonitor:
        plugin: "roadnet::CollisionMonitor"
        costmap_topic: "global_costmap/costmap_raw" # Do not modify
        rate: 1.0                                # Check frequency (Hz)
        reroute_on_collision: true                 # Trigger replanning when a collision is
detected
        max_cost: 253.0                          # Maximum allowed cost
        check_resolution: 2                       # Check resolution (1 = every cell)
        max_collision_dist: 0.5                   # Maximum check distance (m)

    # Edge cost functions
    # edge_cost_functions: ["CostmapScorer", "DynamicEdgesScorer"]
    edge_cost_functions: ["DynamicEdgesScorer"]
    CostmapScorer:
        plugin: "roadnet::CostmapScorer" # Costmap scorer
        use_maximum: false                # Use the average cost instead of the maximum
cost
        invalid_on_collision: true         # Allow collision edges but compute a high
cost
        invalid_off_map: false             # Do not mark as invalid when outside the
costmap
        max_cost: 253.0                   # Maximum valid cost
        check_resolution: 4               # Check resolution
        costmap_topic: "global_costmap/costmap_raw" # Costmap topic, do not
modify
        weight: 1.0                      # weight
    DynamicEdgesScorer:
        plugin: "roadnet::DynamicEdgesScorer"

```

